

ESTRUCTURAS DE CONTROL

Son el medio por el cual los programadores pueden determinar el flujo de ejecución entre los componentes de un programa.

Se encuentran en dos tipos o niveles

- 1) A Nivel de Unidad: cuando el flujo de control se pasa entre unidades. Intervienen los pasajes de parámetros. Ej una unidad termina y comienza otra o una unidad llama a otra etc.
- 2) A Nivel de Sentencia: se dividen en tres grupos
 - a- Secuencia
 - b- Selección
 - c- Iteración

a- SECUENCIA:

- es el flujo de control mas simple.
- indica la ejecución de una sentencia a continuación de otra.
- el delimitador mas general y mas usado es el “;”
- hay lenguajes que no tienen delimitador y establecen que por cada línea vaya una instrucción. Se los llama orientados a línea, ej Python.
- se pueden agrupar varias sentencias en una, llamada SENTENCIA COMPUESTA, llevan delimitadores.
 - Begin y End en Ada y Pascal.
 - {} en C, C++, Java, etc.

a.1-ASIGNACION(esta dentro de secuencia):

-sentencia que produce cambios en los datos de la memoria. Asigna al l-valor de un objeto de dato el r-valor de una expresión.

A := B	Ej: Pascal, Ada, etc.
A = B	Ej: Fortran, C, Prolog, Python, Ruby, etc.
MOVE B TO A	COBOL
A ← B	APL
(SETQ A B)	LISP

Ej en Python no se modifica el r-valor sino q se apunta a un nuevo objeto con dicho valor

CONCEPTO A TENER EN CUENTA:

Diferencia entre sentencia de asignación y expresión:

-en los lenguajes convencionales existe diferencia, pero en lenguajes como C define a la asignación como una expresión con efectos laterales. C también permite asignación multiple.

-Una expresión es una unidad de código que me devuelve un resultado y la asignación es modificar el valor de una posición de memoria a partir del resultado de la expresión.

-las sentencias de asignación devuelven valores.

b- SELECCIÓN:

- Esta estructura de control permite que el programador pueda expresar una elección entre un cierto numero posible de sentencias alternativas.

If (condición logica) sentencia

-el primero en implementarlo fue Fortran

-ejecuta si es verdadero

-Algol incorporo el "else"

If (cond) then sentencia1 else sentencia2

Problemas:

AMBIGÜEDAD: no establecia por lenguaje como se asociaban los else con los if abiertos, si había mas de un if y habia un else no se podía establecer a q if correspondia ese else (PL/1, Pascal y C no eran ambiguos, estos establecieron que cada else era para el ultimo if abierto)

Solucion:

Sentencias de cierre del bloque condicional ej end if.

Desventajas:

Ilegibilidad: programas con muchos if anidados son ilegibles. Utilizar Begin y End .

Respetar identacion.

SELECCIÓN – EJEMPLO EN C

```
int main ()
{
    int year = 0;
    printf (" Introduzca el año : ");
    scanf ("%d", & year);
    if ((0 == year % 400) || ((0 == year % 4) && (0 != year % 100)))
        printf ("El año %d es bisiestro \n", year);
    else
        printf ("El año %d NO es bisiestro \n", year);
    return 0;
}
```

No lleva la palabra clave "then" cómo en Pascal

Si son más de una sentencia se deben encerrar entre { }

(i<j?z:y) Otra forma de Selección que permite C

SELECCIÓN

If then else de Python, sin Ambigüedad y legible

```
if sexo == 'M':  
    print 'La persona es Mujer'  
else :  
    print 'La persona es de otro género'
```

```
if hora <= 6 and hora >=12:  
    print 'Buenos días!!'  
elif hora >12 and hora < 20 :  
    print 'Buenas tardes!!'  
else :  
    print 'Buenas noches!!'
```

- : es obligatorio al final del if, else y del elif
- La **indentación** es **obligatoria** al colocar las sentencias correspondientes tanto al if, else y el elif

Las reglas sintácticas de Python fueron hechas para no permitir ambigüedad y mayor legibilidad. Identación obligatoria.

Python tiene sentencia condicional corta

Sentencia condicional – A if C else B de Python

- Construcción equivalente al “?” del lenguaje C
- Devuelve A si se cumple la condición C, sinó devuelve B

Ejemplo de uso:

```
>>> altura = 1.79  
>>> estatura = "Alto" if altura > 1.65 else "Bajo"  
>>> estatura  
'Alto'  
>>>
```

IMPORTANTE: Python para evaluar las condiciones utiliza la “Evaluación con circuito corto”

FORMAS DE EVALUAR EXPRESION LOGICA:

-Circuito corto: cuando se evalúa la primera parte y si la segunda condición no es necesaria no se evalúa. Permite ahorrar tiempo si es computacionalmente costoso evaluar todas las condiciones y permite evitar errores por ejemplo no evaluar una condición si el objeto es null. Ej if (a != null) and (a.nombre == “Agostina”). En este caso si A es null no se evalúa la segunda condición q sería un error.

-Circuito largo: evalúa todas las condiciones

SELECCIÓN MULTIPLE (dentro de selección)

-sentencia Select, más seguro que el if.

-PL/1 incorpora la sentencia de selección entre dos o más opciones

```

Select
    when(A) sentencia1;
    when(B) sentencia2
    .....
    Otherwise sentencia n;
End;

```

Se le pone etiquetas a la sentencias a ejecutar y el otherwise es para cualquier otro valor

Pascal en vez de Select usa Case

-incorpora q los valores de la expresión sean ordinales y ramas con etiquetas. No importa el orden en el que aparecen las ramas y tiene la opción ELSE

-es inseguro porq no establece q sucede cuando un valor no cae dentro de las alternativas puestas ya que el else NO es obligatorio

```

begin
    readln(opcion);
case opcion of
    '1' : nuevaEntrada;
    '2' : cambiarDatos;
    '3' : borrarEntrada
        else writeln('Opcion no valida!!')
end
end

```

En Ada

Se utiliza un case is

Las expresiones pueden ser solamente de tipo entero o enumerativas

En las selecciones se debe estipular todos los valores posibles q puede tomar la expresión

Tiene la clausula Others q se debe utilizar para representar a aquellos valores q no se especificaron explícitamente.

Ejemplo 1

Case Operator is

```
when '+' => result:= a + b;  
when '-' => result:= a - b;  
when others => result:= a * b;
```

La cláusula **others** se debe colocar porque las etiquetas de las ramas NO abarcan todos los posibles valores de Operator

end case

Ejemplo 2

Case Hoy is

```
when MIE..VIE => Entrenar_duro; -- Rango.  
when MAR | SAB => Entrenar_poco; -- Varias elecciones.  
when DOM => Competir; -- Una elección.  
when others => Descansar; -- Debe ser única y la última alternativa.  
end case;
```

EN C y C++

Usa Switch

Cada rama es etiquetada por uno o mas valores constantes

Cuando la opción coincide con una etiqueta del Switch se ejecutan las sentencias asociadas y se continua con las sentencias de las otras entradas

Existe break q provoca la salida

Tiene una clausula default q sirve para los casos q el valor no coincida con ninguna de las opciones establecidas.

```
Switch Operator {  
    case '+' :  
        result:= a + b; break;  
    case '-' :  
        result:= a - b; break;  
    default :  
        result:= a * b;  
}
```

De be ponerse la sentencia **break** para saltar las siguientes ramas

RUBY

```
raza = 'enano'
# Ahora utilizaremos el case
puts 'Utilizando case asignado a una variable : '
personaje = case raza
              when 'elfo' then 'Legolas'
              when 'enano' then 'Gimli'
              when 'mago' then 'Gandalf'
              when 'ents' then 'Barbol'
              when 'humano' then 'Aragorn'
              when 'orco' then 'Ufthak'
            end
puts personaje
```

Si no entra a ninguno la variable queda sin valor

c- ITERACIÓN:

-Este tipo de instrucciones se utilizan para representar aquellas acciones que se repiten un cierto numero de veces

Evolucion:

Fortran lo definio como un DO Etiqueta variableDeControl

Sentencia Do de Fortran	Ejemplo:
Do label var-de-control= valor1, valor2	DO 1 I = 1,10
.....	SUM = SUM A(I)
Label continue	1 CONTINUE

Con el label continue hacia q vuelva a ejecutarse.

-la variable de control solo puede tomar valores enteros

-siempre una vez se ejecutaba porq evaluaba que haya llegado al limite al final del bucle

En Pascal, ADA, C, C++

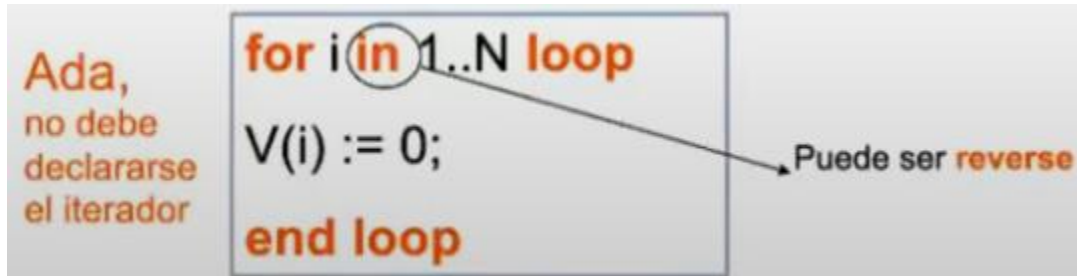
Sentencia for

La variable de control puede tomar cualquier valor ordinal, no solo enteros

Pascal estándar no permite q se toquen ni los valores del limite inferior ni los del superior, ni el valor de la variable de control

El valor de la variable de control fuera del bloque se asume indefinida en Pascal (inseguridad)

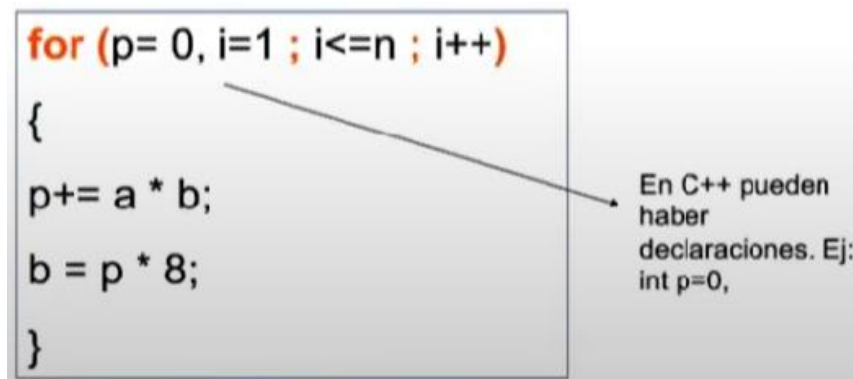
Es inseguro si tocamos el iterador en otra unidad porq fuera de la unidad del for si se puede modificar el valor de la variable



EN C y C++

-tres partes: inicialización, y dos expresiones, la primera (2do parametro) es el testeo q se realiza ANTES de cada iteración. Si no se coloca el for queda en LOOP

- en el primer y ultimo parámetro se pueden poner sentencias separadas por “,”



PYTHON

Estructura q permite iterar sobre una secuencia, es decir, lista, tupla, etc.

```
lista = [ "el", "for", "recorre", "toda", "la", "lista"]
for variable in lista:
    print variable
```

Imprimirá:

- el
- for
- recorre
- toda
- la
- lista

Si se quiere que el for actúe como en los demás lenguajes, entonces hacer uso de la función "range()", ej. range(6). Hará que variable tome los valores de 0 a 5.

21

Python, uso de la función "range()"

- La función devuelve una lista de números enteros ej. range(5), devuelve [0,1,2,3,4].
- Puede tener hasta 3 argumentos:
 - 2 argumentos: range(2,5), devuelve [2,3,4]
 - 3 argumentos: range(2,5,2), devuelve [2,4]

Esa función range() da la posibilidad de simular la sentencia FOR de otros lenguajes

```
for i in range (valor- inicial, valor-
final + 1): acciones
```

Si al range le mando tres argumentos, el ultimo dice de cuanto avanza, en el ejemplo avanza de 2 en 2.

```
for vuelta in range(1,10):
    print("Vuelta "+str(vuelta))
```

Salida

```
Vuelta 1
Vuelta 2
Vuelta 3
....
Vuelta 10
```


WHILE(pertenece a iteracion):

-permite la repetir un proceso mientras se cumpla la condición q se evalua antes de entrar

- En **Pascal**:

sentencia;

- En **C, C++**:

- En **ADA**:

- En **Phyton**:

sentencia1

sentencia2

while condición do

while (condición) sentencia;

**while condición sentencia
end loop;**

while condición :

31

UNTIL(dentro de iteracion):

Estructura q permite repetir un proceso mientras se cumpla una condición la cual se evalua al final por ende una vez se ejecuta como mínimo.

- En **Pascal**:

repeat

sentencia

until condición;

- En **C, C++**:

do sentencia;

while (condición);

LOOP(dentro de iteracion): iteración continua o indefinida

Ada tiene una estructura iterativa

```
loop
...
end loop;
```

De este bucle se sale normalmente, mediante una sentencia "**exit when**" o con una alternativa que contenga una cláusula "**exit**".

```
loop
...
exit when condición;
...
end loop;
```

Debo provocar la salida.